PRACTICAL 5
CREATE DOCKER IMAGES AND CONTAINERS AND RUN CONTAINERIZED APPLICATIONS

Practical objective:
In this practical, you will create a simple application, write a Dockerfile, build a Docker image, run a Docker container, test the containerized application, inspect images and containers, stop/remove containers, and clean up Docker resources safely.

Recommended platform used:
Google Cloud VM with Docker Engine installed.

Main tools used:
- Google Cloud Compute Engine VM
- Docker Engine on the VM
- Docker CLI
- SSH terminal
- Text editor such as nano or vim inside the VM
- Web browser only for opening Google Cloud Console

Estimated outcome:
By the end of this practical, you will be able to:
- Create or use a Google Cloud VM as a Docker lab machine.
- Install Docker Engine on the VM if it is not already installed.
- Explain Docker image and Docker container.
- Create a simple application.
- Create a Dockerfile.
- Create a .dockerignore file.
- Build a Docker image.
- Run a container from an image.
- Map host port to container port.
- View logs.
- Execute commands inside a container.
- Stop and remove containers.
- Remove images after the practical.
- Review Docker configuration using current container best practices.

Important safety message:
Docker containers are powerful. Do not run unknown images from untrusted sources. Do not put passwords, API keys, private keys, or secrets inside Dockerfiles or container images.

Lab environment recommendation:
For classroom labs, it is better to perform this practical on a cloud VM instead of installing Docker Desktop on every local lab PC. You only need browser access to Google Cloud Console and SSH access to the VM. This avoids software installation permission issues on many computers and gives every student a similar Linux-based Docker environment.

1. BASIC THEORY BEFORE STARTING

1.1 What is Docker?

Docker is a platform used to build, run, and manage containers.

Docker helps package an application with its dependencies so it can run consistently on different systems.

Docker solves the common problem where an application works on one computer but fails on another due to missing dependencies or different runtime versions.

1.2 What is a container?

A container is a lightweight isolated runtime environment that runs an application and its dependencies.

1.3 What is a Docker image?

A Docker image is a read-only package used to create containers.

It may contain:
- Application code
- Runtime
- Libraries
- Configuration
- Startup command

1.4 What is a Dockerfile?

A Dockerfile is a text file containing instructions to build a Docker image.

Common instructions:
- FROM
- WORKDIR
- COPY
- RUN
- EXPOSE
- USER
- CMD

Docker reads the Dockerfile step by step and creates an image.

1.5 What is .dockerignore?

.dockerignore is a file that tells Docker which files and folders should not be sent to the build context.

Examples:
- .git
- node_modules
- .env
- secret files
- temporary files

.dockerignore helps keep images cleaner and prevents accidentally sending sensitive or unnecessary files into the build process.

1.6 What is port mapping?

Port mapping connects a port on the VM host to a port inside the container.

Example:

```
-p 8080:3000
```

Meaning:
- Host port 8080 maps to container port 3000.

Without port mapping, a web application running inside a container may not be reachable from your browser.

1.7 What is a non-root container user?

By default, some containers may run as root. A safer practice is to run the application as a non-root user inside the container.

Running as non-root reduces risk if an application is compromised.

2. REQUIREMENTS

2.1 Hardware requirements

- Computer or laptop for accessing Google Cloud Console
- Internet connection

2.2 Software requirements

- Web browser on the local machine
- Google Cloud VM running Debian or Ubuntu
- Docker Engine installed on the VM
- Text editor inside the VM such as nano or vim

Local alternative:
Docker Desktop on Windows or macOS can also be used, but for a college lab with many PCs, the VM-based method is easier to manage because it avoids installing Docker Desktop on every machine.

2.3 Knowledge requirements

- Basic command-line usage
- Basic understanding of ports
- Basic understanding of files and folders

In this practical, Docker commands should be run inside the Google Cloud VM SSH terminal, not on the local lab PC, unless your teacher specifically chooses the local Docker Desktop method.

3. STEP-BY-STEP PRACTICAL

STEP 1: Create or open a Google Cloud VM

Use the VM created in Practical 2, or create a small Debian/Ubuntu VM for this Docker practical.

Recommended lab VM:
- Machine type: e2-small or e2-medium
- Operating system: Debian or Ubuntu
- Disk: 10 GB or more
- Firewall: Do not open extra ports unless required

The VM will act as your Docker lab computer. This means Docker does not need to be installed on your local classroom PC.

Expected result:
A Linux VM is available and running.

STEP 2: Connect to the VM using SSH

Open the VM from Google Cloud Console and click:

SSH

All Docker commands in this practical should be typed inside this SSH terminal.

Expected result:
You are connected to the VM terminal.

STEP 3: Update package information on the VM

Run:

```
sudo apt update
```

This refreshes the package list before installing Docker Engine.

Expected result:
Package information is updated.

STEP 4: Install Docker Engine on the VM if needed

If Docker is not installed, install Docker Engine using Docker's official Debian repository method.

Run these commands on a Debian VM:

```
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/debian/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/debian
Suites: $(. /etc/os-release && echo "$VERSION_CODENAME")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

This installs Docker Engine directly on the Linux VM. Using the official Docker repository provides current Docker packages such as Docker Engine, CLI, containerd, Buildx, and Compose plugin.

Expected result:
Docker Engine is installed on the VM.

STEP 5: Check Docker installation

Inside the VM SSH terminal, run:

```
docker --version
```

Expected result:
Docker version is displayed.

STEP 6: Check Docker is running

Run:

```
sudo docker info
```

Expected result:
Docker system information is displayed.

STEP 7: Optional permission setup for lab convenience

If your teacher allows running Docker without typing sudo every time, run:

```
sudo usermod -aG docker $USER
```

Then exit and reconnect to SSH.

Adding your user to the docker group allows Docker commands without sudo, but it also gives powerful control over Docker on the VM. Use it only on lab VMs where your teacher allows it.

Expected result:
After reconnecting, Docker commands can run without sudo. If this step is not done, use sudo before Docker commands.

STEP 8: Test Docker with hello-world

Run:

```
sudo docker run hello-world
```

This verifies that Docker can download and run a test container successfully.

Expected result:
Docker prints a hello-world success message.

STEP 9: Create a practical folder

Create a folder named:

cloudnative-docker-practical

Open this folder in your VM terminal:

```
mkdir cloudnative-docker-practical
cd cloudnative-docker-practical
```

Keeping all practical files in one folder makes the build context clear and easy to clean up.

Expected result:
You are inside the practical folder.

STEP 10: Create package.json

Create a file named:

package.json

Add this content:

```
{
  "name": "cloudnative-docker-practical",
  "version": "1.0.0",
  "description": "Simple Docker practical application",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {}
}
```

package.json describes a Node.js application and defines how to start it.

Expected result:
package.json is created.

STEP 11: Create server.js

Create a file named:

server.js

Add this content:

```
const http = require("http");

const port = process.env.PORT || 3000;

const server = http.createServer((req, res) => {
  res.writeHead(200, { "Content-Type": "text/plain" });
  res.end("Hello from Cloud Native Docker Practical 5\n");
});

server.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

This is a very small web server. It listens on port 3000 and returns a simple message in the browser.

Expected result:
server.js is created.

STEP 12: Create .dockerignore

Create a file named:

.dockerignore

Add this content:

```
.git
node_modules
npm-debug.log
.env
*.pem
*.key
Dockerfile
README.md
```

This file prevents unnecessary or sensitive files from being sent to Docker during image build.

Expected result:
.dockerignore is created.

STEP 13: Create Dockerfile

Create a file named:

Dockerfile

Add this content:

```
FROM node:22-alpine

WORKDIR /app

COPY package.json ./

RUN npm install --omit=dev

COPY server.js ./

RUN addgroup -S appgroup && adduser -S appuser -G appgroup

USER appuser

EXPOSE 3000

CMD ["npm", "start"]
```

This Dockerfile uses a small Node.js Alpine base image, copies the application,

creates a non-root user, exposes port 3000, and starts the server.

Expected result:
Dockerfile is created.

STEP 14: Understand the Dockerfile instructions

Review each instruction:

FROM:
Selects the base image.

WORKDIR:
Sets working directory inside the image.

COPY:
Copies files into the image.

RUN:
Runs commands while building the image.

USER:
Runs the application as a non-root user.

EXPOSE:
Documents the port used by the container.

CMD:
Defines the default startup command.

Understanding each instruction helps you debug image build problems.

Expected result:
You can explain the purpose of each Dockerfile instruction.

STEP 15: Build the Docker image

Run:

```
sudo docker build -t cloudnative-practical-5:1.0 .
```

Expected result:
Docker image is built successfully.

STEP 16: List Docker images

Run:

```
sudo docker images
```

Expected result:
cloudnative-practical-5 with tag 1.0 appears in the image list.

STEP 17: Run the container

Run:

```
sudo docker run --name cloudnative-app -p 8080:3000 cloudnative-practical-5:1.0
```

Expected result:
The application starts and shows a message such as "Server running on port 3000" in the terminal.

STEP 18: Test the application from inside the VM

Open another SSH terminal or stop the foreground container later and test in detached mode. If the container is running in the current terminal, use a second SSH session and run:

```
curl http://localhost:8080
```

Because the container is running on the VM, localhost means the VM itself. This test does not require opening a public firewall port.

Expected result:
The terminal displays:

```
Hello from Cloud Native Docker Practical 5
```

STEP 19: Optional browser test from local machine

If your teacher specifically wants browser testing from the local PC, create a temporary firewall rule that allows TCP port 8080 only from the required source IP range. Then open:

```
http://VM_EXTERNAL_IP:8080
```

Do not open port 8080 to the whole internet unless your teacher instructs you. Testing with curl inside the VM is safer for a beginner lab.

Expected result:
The browser displays the application message only if firewall access is allowed.

STEP 20: Stop the running container

In the terminal where the container is running, press:

```
Ctrl + C
```

This stops the running container process.

Expected result:
The container stops.

STEP 21: Run container in detached mode

Run:

sudo docker run -d --name cloudnative-app -p 8080:3000 cloudnative-practical-5:1.0

Detached mode runs the container in the background so your terminal remains available.

Expected result:
Docker prints a container ID and returns control to the terminal.

STEP 22: List running containers

Run:

sudo docker ps

docker ps shows containers that are currently running.

Expected result:
cloudnative-app appears in the running container list.

STEP 23: View container logs

Run:

sudo docker logs cloudnative-app

Logs help you see application output and troubleshoot problems.

Expected result:
The server startup log appears.

STEP 24: Execute a command inside the container

Run:

sudo docker exec cloudnative-app node --version

docker exec runs a command inside a running container. This command checks the Node.js version inside the container.

Expected result:
Node.js version is displayed.

STEP 25: Inspect container configuration

Run:

sudo docker inspect cloudnative-app

docker inspect shows detailed JSON information about the container, including image, network, ports, mounts, and configuration.

Expected result:
Container configuration details are displayed.

STEP 26: Stop the detached container

Run:

```
sudo docker stop cloudnative-app
```

This stops the background container gracefully.

Expected result:
The container stops.

STEP 27: List all containers

Run:

```
sudo docker ps -a
```

Expected result:
cloudnative-app appears as a stopped container.

STEP 28: Remove the stopped container

Run:

```
sudo docker rm cloudnative-app
```

Stopped containers still exist on your system until you remove them.

Expected result:
The stopped container is removed.

STEP 29: Run container with automatic cleanup

Run:

```
sudo docker run --rm --name cloudnative-app-temp -p 8080:3000 cloudnative-practical-5:1.0
```

Use curl http://localhost:8080 from another SSH session to test.

Then press Ctrl + C.

The --rm flag removes the container automatically after it stops. This is useful for temporary test containers.

Expected result:
The app runs and the container is removed automatically after stopping.

STEP 30: Tag the image with another tag

Run:

```
sudo docker tag cloudnative-practical-5:1.0 cloudnative-practical-5:latest
```

Tags are labels for image versions. For real production deployments, avoid depending only on latest because it can change.

Expected result:
The image has both 1.0 and latest tags.

STEP 31: Check image history

Run:

```
sudo docker history cloudnative-practical-5:1.0
```

Image history shows layers created during the build. It helps you understand how Dockerfile instructions become image layers.

Expected result:
Image layer history is displayed.

STEP 32: Optional image scan

If Docker Scout is available, run:

```
sudo docker scout quickview cloudnative-practical-5:1.0
```

or use another approved scanner such as Trivy if installed:

```
trivy image cloudnative-practical-5:1.0
```

Image scanning checks known vulnerabilities in images. In modern CI/CD pipelines, image scanning is commonly automated.

Expected result:
Scanner output appears if the tool is installed.

STEP 33: Remove image tags

After completing the practical, run:

```
sudo docker rmi cloudnative-practical-5:latest
sudo docker rmi cloudnative-practical-5:1.0
```

Removing images frees disk space. Make sure no container is using the image before removing it.

Expected result:
The image tags are removed.

STEP 34: Check Docker disk usage

Run:

```
sudo docker system df
```

Expected result:
Docker disk usage summary appears.

STEP 35: Clean up carefully

For this lab only, if your teacher allows cleanup, run:

sudo docker builder prune

Prune commands remove unused Docker data. Be careful with prune commands on shared lab machines or production systems because they can remove resources needed by others.

Expected result:
Unused build cache is cleaned if confirmed.

4. OPTIONAL DOCKER COMMAND SUMMARY

Useful commands:

Note:
If your VM user is not added to the docker group, run these commands with sudo.

docker --version
- Show Docker version.

docker info
- Show Docker system information.

docker build -t image-name:tag .
- Build an image.

docker images
- List images.

docker run image-name:tag
- Run a container.

docker run -d image-name:tag
- Run a container in background.

docker ps
- List running containers.

docker ps -a
- List all containers.

docker logs container-name
- View container logs.

docker exec container-name command
- Run command inside container.

docker stop container-name
- Stop a running container.

docker rm container-name

- Remove stopped container.

docker rmi image-name:tag
- Remove image.

docker system df
- Show Docker disk usage.

5. OBSERVATION TABLE

Write your observations in this format:

VM name:

VM operating system:

Docker version:

Application folder name:

Image name:

Image tag:

Container name:

Host port:

Container port:

curl output from VM:

Browser output, only if optional firewall access was configured:

Command used to build image:

Command used to run container:

Was .dockerignore created?

Was non-root USER used?

Was the image scanned?

Were containers removed?

Were images removed?

6. RESULT

A simple Node.js application was containerized using Docker. A Dockerfile and .dockerignore file were created, a Docker image was built, a container was run, the application was tested from inside the VM using curl, logs and container details were viewed, and Docker resources were cleaned up after the practical.

7. CONCLUSION

This practical covered the complete beginner workflow for Docker: prepare a cloud VM, install Docker Engine, create application files, write a Dockerfile, build an image, run a container, test the application, inspect logs and configuration, and clean up resources. They also learned important container best practices such as using .dockerignore, running as non-root, avoiding secrets in images, using explicit tags, scanning images, and cleaning up carefully.

8. LATEST-PRACTICES REVIEW TEST

This review checks whether the practical follows current Docker and container best practices.

1. Was the practical performed on a controlled VM instead of requiring Docker Desktop on every lab PC?
   Yes. A cloud VM gives a consistent Linux Docker environment and avoids installing Docker Desktop on many classroom machines.

2. Was Docker Engine installed from an official Docker repository or provided by an approved lab image?
   Yes. Docker Engine should come from an approved source instead of unknown install scripts or untrusted packages.

3. Was a trusted base image used?
   Yes. The practical uses the official Node.js Alpine image.

4. Was a smaller base image selected?
   Yes. Alpine-based images are smaller than full operating system images and help reduce image size.

5. Was .dockerignore created?
   Yes. .dockerignore reduces build context and helps prevent accidental inclusion of unnecessary or sensitive files.

6. Were secrets avoided in the Dockerfile and image?
   Yes. No passwords, API keys, private keys, or .env values should be copied into the image.

7. Was the container configured to run as a non-root user?
   Yes. The Dockerfile creates appuser and uses the USER instruction.

8. Was an explicit image version tag used?
   Yes. The image is built as cloudnative-practical-5:1.0. The latest tag is shown only for learning and should not be the only production reference.

9. Was the container treated as ephemeral?
   Yes. The lab demonstrates --rm for temporary containers and removes stopped

containers.

10. Was image scanning introduced?
   Yes. Docker Scout or Trivy scanning is included as an optional step.

11. Were cleanup commands used carefully?
   Yes. Containers and images are removed explicitly, and prune is shown with a caution.

12. Was public firewall access avoided unless needed?
   Yes. The main test uses curl inside the VM. Public browser access should only be used when a temporary firewall rule is intentionally configured.

13. Would a production image use only this simple Dockerfile?
   Not always. Production images may use multi-stage builds, pinned digests, dependency lock files, SBOM/provenance, registry scanning, signing, and stricter runtime security controls.

If the points above are covered, the practical aligns with current platform practices.
practices and introduces current image-building and container-running recommendations.

9. VIVA QUESTIONS

1. What is Docker?
2. What is a Docker image?
3. What is a Docker container?
4. What is a Dockerfile?
5. What is .dockerignore?
6. What is the purpose of FROM instruction?
7. What is the purpose of COPY instruction?
8. What is the purpose of EXPOSE instruction?
9. What is the purpose of CMD instruction?
10. Why should containers avoid running as root?
11. What is port mapping?
12. What is the difference between docker ps and docker ps -a?
13. Why should secrets not be stored in images?
14. Why should images be scanned?

10. SHORT ANSWERS FOR VIVA

1. Docker is a platform for building, running, and managing containers.
2. A Docker image is a read-only package used to create containers.
3. A Docker container is a running instance of an image.
4. A Dockerfile contains instructions to build a Docker image.
5. .dockerignore excludes files from the Docker build context.
6. FROM selects the base image.
7. COPY copies files into the image.
8. EXPOSE documents the port used by the container.
9. CMD defines the default command to run when the container starts.
10. Running as non-root reduces security risk.

11. Port mapping connects a host port to a container port.
12. docker ps shows running containers; docker ps -a shows all containers.
13. Secrets in images can be leaked through image layers or registries.
14. Image scanning detects known vulnerabilities.

11. COMMON MISTAKES TO AVOID

1. Running Docker commands on the local PC instead of inside the VM SSH terminal.
2. Forgetting to start Docker before running commands.
3. Running docker build from the wrong folder.
4. Forgetting the dot at the end of docker build command.
5. Using latest as the only image tag.
6. Copying secrets into the image.
7. Not creating .dockerignore.
8. Running containers as root without need.
9. Forgetting port mapping.
10. Opening public firewall access unnecessarily.
11. Leaving containers running after the practical.
12. Removing images before stopping containers.
13. Running aggressive prune commands on shared systems.
14. Downloading and running untrusted images.

12. PRACTICAL COMPLETION CHECKLIST

Before submitting this practical, confirm:

1. You created or opened a Google Cloud VM.
2. You connected to the VM using SSH.
3. You installed Docker Engine on the VM or confirmed it was already installed.
4. You checked Docker version.
5. You checked Docker engine status.
6. You created the practical folder inside the VM.
7. You created package.json.
8. You created server.js.
9. You created .dockerignore.
10. You created Dockerfile.
11. You understood Dockerfile instructions.
12. You built the Docker image.
13. You listed Docker images.
14. You ran the container.
15. You tested the application using curl inside the VM.
16. You avoided public firewall access unless instructed.
17. You stopped the container.
18. You ran the container in detached mode.
19. You viewed running containers.
20. You viewed container logs.
21. You executed a command inside the container.
22. You inspected the container.
23. You removed stopped containers.
24. You tested --rm cleanup.
25. You tagged the image.
26. You checked image history.
27. You introduced image scanning.
28. You removed image tags after the practical.
29. You checked Docker disk usage.
30. You completed the latest-practices review test.
31. You answered the viva questions.